Machine Learning Laboratory

Experiment No. 10

MLP on Wine Quality Dataset

Aim

Implement and train a Multilayer Feed-Forward Network (MLP) on the Wine Quality dataset. Experiment with different numbers of hidden layers and neurons, and analyze how these architectural choices affect the network's performance.

Objectives

- Load and preprocess the Wine Quality dataset.
- Convert the wine quality score into a binary classification problem.
- Design and implement an MLP with different architectures.
- Experiment with different numbers of hidden layers and neurons.
- Train and evaluate each neural network.
- Compare the classification accuracy of different architectures.
- Analyze the effect of network architecture on performance.

Description

A Multilayer Perceptron (MLP) is a feed-forward neural network consisting of an input layer, one or more hidden layers, and an output layer. In this experiment, an MLP is implemented to classify wines based on their quality.

The Wine Quality dataset contains 11 input features representing different chemical properties of wine. The original quality score is converted into a binary classification label.

Wines with a quality score greater than or equal to 6 are classified as Good Wine (1), while wines with a quality score less than 6 are classified as Bad Wine (0).

The classification label is defined as:

$$y = \begin{cases} 1, & \text{if quality} \geq 6 \\ 0, & \text{if quality} < 6 \end{cases}$$

The input features are standardized before training using the StandardScaler method.

The MLP consists of an input layer, hidden layers, and an output layer. The hidden layers use the ReLU activation function, while the output layer uses the Softmax activation function.

Theory

For a neuron, the weighted sum of inputs is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

where w_i represents the weights, x_i represents the input features, and b represents the bias.

The output of a neuron is obtained by applying an activation function:

$$a = f(z)$$

In this experiment, the ReLU activation function is used in the hidden layers.

The ReLU activation function is given by:

$$f(z) = \max(0, z)$$

The output layer uses the Softmax activation function to classify the input into two classes.

The Softmax function is given by:

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

where $K = 2$ represents the number of output classes.

Data Preprocessing

The Wine Quality dataset is loaded from a CSV file.

The original quality column is converted into a binary target label. The input features are separated from the target variable and standardized using:

$$X' = \frac{X - \mu}{\sigma}$$

where μ is the mean of the feature and σ is its standard deviation.

The dataset is divided into training and testing sets using an 80:20 ratio.

The target labels are converted into one-hot encoded form before training.

MLP Architectures

Three different MLP architectures are implemented and compared.

Model 1: One Hidden Layer

The first model contains one hidden layer with 10 neurons.

$$11 \to 10 \to 2$$

Here:

- 11 = input features
- 10 = neurons in the hidden layer
- 2 = output classes

Model 2: Two Hidden Layers

The second model contains two hidden layers with 20 and 10 neurons.

$$11 \to 20 \to 10 \to 2$$

Here:

- 11 = input features
- 20 = neurons in the first hidden layer
- 10 = neurons in the second hidden layer
- 2 = output classes

Model 3: Three Hidden Layers

The third model contains three hidden layers with 20, 10, and 5 neurons.

$$11 \to 20 \to 10 \to 5 \to 2$$

Here:

- 11 = input features
- 20 = neurons in the first hidden layer
- 10 = neurons in the second hidden layer
- 5 = neurons in the third hidden layer
- 2 = output classes

Algorithm

Step 1: Start.

Step 2: Import the required libraries such as Pandas, Scikit-learn, and TensorFlow/Keras modules.

Step 3: Load the Wine Quality dataset from the local CSV file.

Step 4: Create the classification label:

$$y = \begin{cases} 1, & \text{if quality} \geq 6 \\ 0, & \text{if quality} < 6 \end{cases}$$

Step 5: Separate the input features X and target labels y.

Step 6: Remove the original *quality* and *quality_label* columns from the input features.

Step 7: Standardize the input features using StandardScaler:

$$X' = \frac{X - \mu}{\sigma}$$

Step 8: Split the dataset into training and testing sets using an 80:20 ratio.

Step 9: Convert the target labels into one-hot encoded form.

Step 10: Create the first MLP with one hidden layer containing 10 neurons.

The architecture is:

$$11 \rightarrow 10 \rightarrow 2$$

Step 11: Apply ReLU activation to the hidden layer and Softmax activation to the output layer.

Step 12: Compile the network using the Adam optimizer, categorical cross-entropy loss function, and accuracy as the evaluation metric.

Step 13: Train the first MLP for 50 epochs with a batch size of 16.

Step 14: Evaluate the trained first MLP using the testing dataset and record its classification accuracy.

Step 15: Create the second MLP with two hidden layers containing 20 and 10 neurons respectively.

The architecture is:

$$11 \rightarrow 20 \rightarrow 10 \rightarrow 2$$

Step 16: Apply ReLU activation to the hidden layers and Softmax activation to the output layer.

Step 17: Compile the second MLP using the Adam optimizer, categorical cross-entropy loss, and accuracy metric.

Step 18: Train the second MLP for 50 epochs with a batch size of 16.

Step 19: Evaluate the second MLP using the testing dataset and record its classification accuracy.

Step 20: Create the third MLP with three hidden layers containing 20, 10, and 5 neurons respectively.

The architecture is:

$$11 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2$$

Step 21: Apply ReLU activation to the hidden layers and Softmax activation to the output layer.

Step 22: Compile the third MLP using the Adam optimizer, categorical cross-entropy loss, and accuracy metric.

Step 23: Train the third MLP for 50 epochs with a batch size of 16.

Step 24: Evaluate the third MLP using the testing dataset and record its classification accuracy.

Step 25: Compare the classification accuracies of the three MLP architectures.

Step 26: Analyze the effect of increasing the number of hidden layers and neurons on network performance.

Step 27: Display the comparison results.

Step 28: Stop.

Performance Metric

The performance of the MLP models is evaluated using classification accuracy.

The classification accuracy is calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100$$

The three models are compared using:

$$\text{Accuracy}_{Model\,1}$$

$$\text{Accuracy}_{Model\,2}$$

$$\text{Accuracy}_{Model\,3}$$

The model with the highest test accuracy provides the best classification performance for the given experiment.

Advantages and Disadvantages

Advantages

- MLP can learn complex relationships between input features.

- Multiple hidden layers provide greater representation capacity.
- ReLU provides efficient nonlinear learning.
- Different architectures can be compared to identify a suitable network structure.
- MLP can be used for classification problems with multiple input features.

Disadvantages

- Increasing the number of hidden layers increases computational complexity.
- A network with too many neurons may overfit the training data.
- Deeper networks may require more training time.
- The performance depends on the choice of architecture and training parameters.

Expected Outcome

- The Wine Quality dataset is successfully loaded and preprocessed.
- The wine quality problem is converted into a binary classification problem.
- Three MLP architectures are successfully implemented.
- The models are trained and evaluated on the testing dataset.
- The classification accuracy of the three architectures is compared.
- The effect of increasing hidden layers and neurons on network performance is analyzed.
- The architecture giving the best test performance can be identified.

Result

Three Multilayer Perceptron networks with different numbers of hidden layers and neurons were implemented and trained on the Wine Quality dataset.

The classification accuracy of the three architectures was evaluated and compared. The effect of changing the number of hidden layers and neurons on the performance of the neural network was analyzed.